

SMILE: Smooth Motion for Improved Long-Horizon VLA Execution

Jongwoo Park¹, E-Ro Nguyen¹, Kanchana Ranasinghe²,
Cristina Mata¹, Xiang Li¹, and Michael S Ryoo¹

¹Stony Brook University ²Salesforce AI Research

jongwopark@cs.stonybrook.edu

Abstract: Vision-Language-Action (VLA) models reduce inference cost by executing multiple actions per call, but longer horizons often degrade accuracy because raw chunks contain jitter and outliers. We introduce SMILE, an architecture-preserving interface that predicts B-spline coefficients and decodes them into smooth action sequences. SMILE changes only the action representation, enabling longer fixed horizons while retaining each baseline’s backbone and model scale. We apply SMILE to SmolVLA, Evo1, VPP, and DAWN, improving accuracy and amortized inference efficiency across LIBERO, CALVIN, and real-world experiments. SMILE-Evo1 reaches 98.0% with a 1.1x speedup on LIBERO, while SMILE-VPP reaches an average length of 4.42 with a 1.5x speedup on CALVIN. At a matched execution horizon of 10, SMILE-SmolVLA reduces non-boundary acceleration by 78.6% and velocity sign-change rate by 42.3%. Real-world xArm tests show higher success, fewer drops, and fewer contacts. These results establish smooth coefficient-space generation as a route to accurate, efficient long-horizon VLA execution.

Project page: jongwoopark7978.github.io/smilevla

I. INTRODUCTION

Vision-Language-Action (VLA) models increasingly rely on action chunking to make closed-loop robot control computationally practical. Rather than invoke a large multimodal policy at every controller step, an action expert predicts a short sequence of low-level controls and the robot executes several actions before refreshing the observation [1], [2], [3], [4]. The number of actions executed per refresh, which we denote by the execution horizon H , directly controls inference efficiency: larger H amortizes one expensive model call over more robot actions.

The benefit of a larger horizon is often offset by a loss in control quality. Raw action chunks can contain high-frequency oscillations, temporally inconsistent directions, and isolated outliers. These errors are tolerable when only one or two predictions are executed, but they accumulate when a longer prefix is executed before replanning. As a result, increasing H can lower amortized latency while simultaneously reducing task success. This behavior is visible for both SmolVLA and Evo1 in Fig. 2: their official raw-action policies become less accurate as more actions are executed per model call. The central question of this work is therefore how to make the

generated action sequence reliable enough that a VLA can safely use a longer fixed horizon.

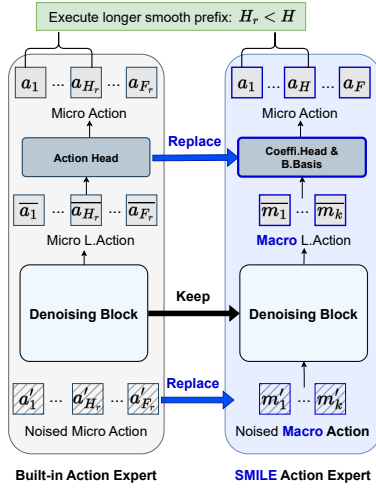
We propose *SMILE*, a smooth hierarchical action interface for denoising VLA action experts. Instead of denoising every low-level action independently in raw action space, SMILE predicts a compact set of B-spline coefficients for translation, rotation, and gripper control. A fixed basis decodes these coefficients into a temporally coherent sequence of execution actions. The coefficient sequence acts as a macro-action representation, while the decoded controls remain compatible with the original robot controller. Because the smoothness is imposed by the representation rather than by a larger backbone or a separate planner, SMILE can increase the execution horizon and reduce amortized latency without increasing the reported model scale.

SMILE is designed as an architecture-preserving replacement for the action-side interface. We retain the original visual-language conditioner and the main denoising backbone, and replace only the lightweight projections and output heads that map between the denoiser and robot actions. This design applies across heterogeneous VLA action experts: VLM-centered SmolVLA and Evo1 [3], [5]; predictive-visual VPP [4]; and explicit pixel-motion-conditioned DAWN [6]. The upstream predictive visual or pixel-motion modules remain unchanged.

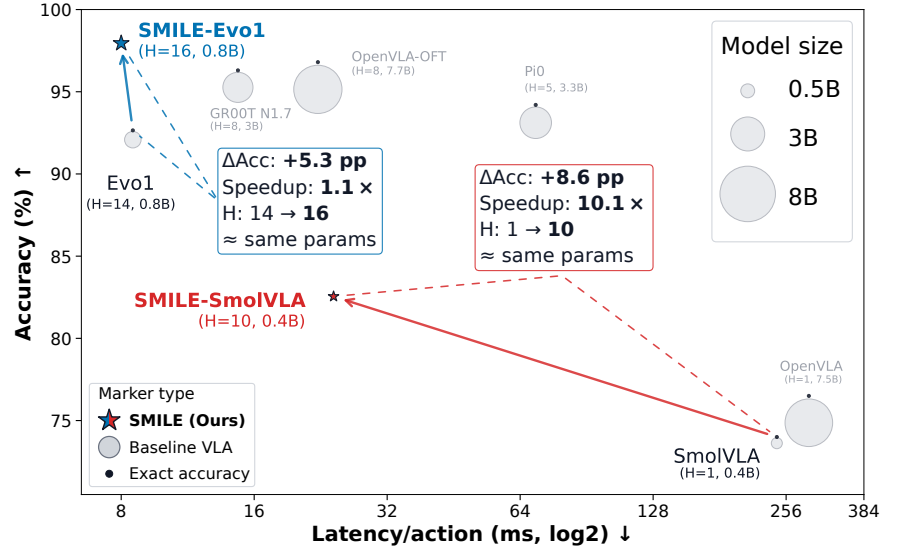
Our experiments show consistent accuracy–efficiency gains across LIBERO, CALVIN, and real-world xArm manipulation. SMILE improves performance across four heterogeneous action experts while enabling longer fixed execution horizons at comparable model-size scales. A matched-horizon analysis further shows substantially lower intra-chunk acceleration and oscillatory action changes, linking these gains to smoother action generation.

Contributions. We make three main contributions:

- 1) We introduce SMILE, a B-spline coefficient action interface that generates smooth action sequences and enables VLAs to execute longer fixed horizons with higher task accuracy and lower amortized inference latency per action. This capability is particularly beneficial for temporally extended manipulation tasks, such as those evaluated in LIBERO and CALVIN.
- 2) We provide an architecture-preserving integration that retains each baseline’s original conditioning pathways and denoising backbone, while maintaining a model



(a) SMILE architecture.



(b) Accuracy-efficiency comparison on LIBERO.

Fig. 1: SMILE architecture and accuracy-efficiency frontier. (a) The built-in action expert directly outputs raw micro-actions $a_1, \dots, a_{F_r}$ and executes a shorter prefix H_r , while SMILE replaces only the action-side raw sequence interface with B-spline coefficient tokens and coefficient heads that decode to smooth micro-actions $a_1, \dots, a_F$, enabling a longer executed prefix $H_r < H$. The original VLA backbone, conditioner, and main denoising action-expert block are retained. (b) Success versus amortized inference latency per action; marker size denotes model scale. SMILE improves the accuracy-efficiency trade-off at comparable model size.

size comparable to that of the corresponding baseline. This interface supports heterogeneous policy designs, including VLM-centered, predictive-visual, and explicit pixel-motion-conditioned action experts.

- 3) We demonstrate consistent gains across SmolVLA, Evo1, VPP, and DAWN on LIBERO, CALVIN, and real-world xArm manipulation, and directly verify that these improvements correlate with substantially smoother fixed-horizon action execution.

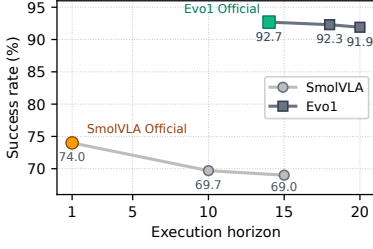
II. RELATED WORK

a) Action chunking and efficient VLA execution: Action chunking predicts a sequence of future controls and executes a prefix before replanning, amortizing policy inference while trading closed-loop reactivity for fewer model refreshes [1], [2], [3], [4]. BID samples and selects candidate chunks at inference time, whereas AAC and AutoHorizon adapt how much of each predicted chunk is executed before replanning [7], [8], [9]. These methods modify chunk selection or execution without changing the learned action representation. SMILE instead reparameterizes the action-side denoising target into compact B-spline coefficients, improving the temporal quality of each generated chunk so that a longer fixed horizon H can be executed reliably. SMILE therefore targets the accuracy-amortized-inference frontier for long fixed-horizon execution, while remaining complementary to candidate sampling and adaptive-horizon selection.

b) Smooth action representations and real-time execution: Action-chunking policies such as ACT and Diffusion Policy predict multi-step controls to improve temporal co-

herence, although raw action chunks can still exhibit intra-chunk jitter and discontinuities across successive policy updates [1], [2]. RTC and ABPolicy primarily address real-time asynchronous execution by generating the next chunk while the current chunk is being executed; RTC conditions the new chunk on previously committed actions, whereas ABPolicy combines a B-spline control-point representation with continuity-aware trajectory updates [10], [11]. A complementary line of work develops compact, structured action representations. FAST compresses action sequences in the frequency domain, BEAST formulates fixed-length B-spline tokens and demonstrates them with a vision-language foundation model and non-VLM policies, and recent spline-based policies directly predict continuous spline trajectories [12], [13], [14], [15]. Within this landscape, SMILE specifically targets synchronous fixed-horizon execution. It provides an architecture-preserving B-spline coefficient interface for heterogeneous compact VLA action experts, retaining their conditioning pathways and main denoising backbones, and explicitly evaluates whether smoother chunks support a longer fixed execution horizon H with higher task accuracy and lower amortized inference latency per action. Thus, SMILE focuses on improving the accuracy-efficiency operating point through reliable long fixed-horizon chunks, rather than primarily addressing inference delay through asynchronous scheduling or introducing a general-purpose action tokenizer.

c) Denoising action experts and structured visual conditioning: A growing class of continuous-action VLAs couples a pretrained vision-language or multimodal backbone with a diffusion- or flow-matching action expert that generates



(a) Success vs execution horizon

	Base	SMILE
<i>Evo1 pair</i>		
H	6	16
DR (%)	8.3	0.0
HR (%)	45.0	22.5
<i>VPP pair</i>		
H	10	15
DR (%)	3.3	0.0
HR (%)	25.0	11.7

(b) Cluttered results.

Fig. 2: Motivation for smooth long-horizon execution. (a) LIBERO success of raw-action baselines across execution horizons; colored markers denote their official default horizons. (b) Selected cluttered xArm results for each baseline–SMILE pair. H denotes the execution horizon while DR and HR denote drop and hit rates; complete results appear in Table III.

complete action chunks [2], [3], [5], [16], [17]. Beyond direct visual-language conditioning, recent policies incorporate dynamics-aware visual structure through predictive video features, compact world-knowledge forecasts, explicit motion intermediates, or joint video-action generation [4], [6], [18], [19], [20], [21]. In our evaluation, VPP conditions an inverse-dynamics policy on predictive representations learned by a video diffusion model, whereas DAWN first generates dense pixel motion and then conditions a low-level diffusion action expert on this explicit motion representation [4], [6]. Together with the VLM-centered SmolVLA and Evo1, these substantially different designs provide heterogeneous testbeds for assessing whether coefficient-space denoising is tied to a particular architecture. SMILE preserves each policy’s original visual-language and motion-conditioning pathways and main denoising backbone, while reparameterizing only the denoised action variable and action-side interface from per-step controls to compact B-spline coefficients. SMILE therefore serves as an architecture-preserving action interface rather than replacing the underlying perceptual or predictive model.

III. OBSERVATIONS ON LONG-HORIZON EXECUTION

We first examine whether simply increasing the execution horizon H , defined as the number of actions executed per model refresh, is sufficient for efficient long-horizon execution. Although a larger H amortizes each model call over more controller steps, it also increases the number of actions executed before the next observation is processed.

Fig. 2 presents two complementary observations. First, the raw-action baselines lose task accuracy as their execution horizons increase, showing that longer execution cannot be obtained reliably by increasing H alone. Second, the SMILE variants operate with longer fixed horizons in the cluttered xArm setting while reducing object drops and contacts with neighboring objects. The lower drop and hit rates are consistent with more stable gripper behavior and more controlled approach and transport motions.

Together, these results motivate improving the temporal structure of the predicted action chunk rather than merely

increasing its execution horizon. SMILE addresses this limitation by denoising compact B-spline coefficients whose decoded actions are coupled across time.

IV. METHOD

A. Overview

At model refresh j , a standard action expert predicts a raw action chunk

$$A_j = [a_{j,1}, \dots, a_{j,F}] \in \mathbb{R}^{F \times d}, \quad (1)$$

where F is the generated chunk length. At inference time, the controller executes only the first $H \leq F$ actions before the next observation is processed [3], [4], [5], [18], [22], [23], [24], [25], [26]. Increasing H reduces amortized inference latency per action, but it also commits the controller to a longer locally open-loop prefix whose individual actions may be temporally inconsistent.

SMILE replaces the raw chunk with a compact macro-action

$$M_j = (\Theta_j^{\text{tr}}, \Theta_j^{\text{rot}}, \Theta_j^{\text{grip}}), \quad (2)$$

consisting of B-spline coefficients for translation, rotation, and gripper control. A deterministic decoder maps M_j to an F -step smooth action sequence that is sent to the original controller, and the controller executes the first H actions. Within each modality, the decoded sequence is jointly determined by a compact set of spline coefficients, coupling neighboring actions across time and reducing sensitivity to isolated predictions. The execution horizon H is fixed within each rollout rather than adapted online.

As shown in Fig. 1a, SMILE retains the computationally dominant components of the baseline, including its VLA backbone, conditioning pathway, and main denoising backbone. Thus, visual-language, predictive-visual, or pixel-motion features are computed as in the original model. We modify only the action-side input projection so that the denoiser operates on noisy coefficient tokens and replace the raw-action output projection with coefficient heads.

B. B-spline action representation and decoding

For a generated chunk of length F , spline degree P , and K control points, let $\Phi_F^{(P,K)} \in \mathbb{R}^{F \times K}$ denote the fixed B-spline basis evaluated at the F uniformly spaced action times. For each chunk start s , the action expert predicts

$$\Theta_s^{\text{tr}} \in \mathbb{R}^{K \times 3}, \quad \Theta_s^{\text{rot}} \in \mathbb{R}^{K \times 3}, \quad \Theta_s^{\text{grip}} \in \mathbb{R}^{K \times 1}. \quad (3)$$

The cumulative translation, cumulative command-space rotation, and continuous gripper trajectories are decoded as

$$\begin{aligned} P_s &= \Phi_F^{(P,K)} \Theta_s^{\text{tr}}, \\ R_s &= \Phi_F^{(P,K)} \Theta_s^{\text{rot}}, \\ G_s &= \text{clip}\left(\Phi_F^{(P,K)} \Theta_s^{\text{grip}}, g_{\min}, g_{\max}\right). \end{aligned} \quad (4)$$

Here, $g_{\min}$ and $g_{\max}$ denote the benchmark- or controller-specific lower and upper bounds for the gripper command.



Fig. 3: SMILE-Evo1 success on LIBERO-Long: Each image shows approach, grasp, lift, and place.

Using the virtual initial cumulative states $P_s(0) = \mathbf{0}$ and $R_s(0) = \mathbf{0}$, translation actions are finite differences of the cumulative path:

$$a_{s,\tau}^{\text{tr}} = P_s(\tau) - P_s(\tau - 1). \quad (5)$$

For rotation, we map the cumulative rotation vectors to $SO(3)$ and take adjacent relative rotations:

$$a_{s,\tau}^{\text{rot}} = \log(\exp(R_s(\tau - 1))^{-1} \exp(R_s(\tau))), \quad (6)$$

which is converted back to the controller’s rotation-vector convention. The gripper command is $a_{s,\tau}^{\text{grip}} = G_s(\tau)$.

The decoded actions $a_{s,\tau}$ are native controller actions; SMILE changes only the representation predicted by the action expert before deterministic decoding. Translation and rotation retain the baseline’s relative-action semantics, while the gripper command remains absolute. At inference time, the controller executes the first $H \leq F$ decoded actions. Unless otherwise stated, all SMILE variants use $K = 8$ control points and cubic splines ($P = 3$), as selected in Table IV.

C. Architecture-preserving denoising objective

SMILE changes the variable denoised by the action expert from a raw action sequence to normalized spline coefficients. Let $\Theta_{s,\text{norm}}^{m,*}$ denote the target coefficient tensor for modality $m \in \{\text{tr}, \text{rot}, \text{grip}\}$, and let c_s denote the baseline’s unchanged multimodal conditioning. For each modality, the baseline’s native corruption process can be written generically as

$$\Theta_{s,t}^m = \alpha_t \Theta_{s,\text{norm}}^{m,*} + \sigma_t \epsilon_m, \quad \epsilon_m \sim \mathcal{N}(0, I). \quad (7)$$

The preserved denoiser predicts the target used by the original training formulation: a velocity field for flow-matching experts or noise for diffusion experts. With target y_t^m , the objective is

$$L_{\text{denoise}}^m = \|f_\theta^m(\Theta_{s,t}, t, c_s) - y_t^m\|_F^2, \quad (8)$$

$$L_{\text{total}} = \lambda_{\text{tr}} L_{\text{denoise}}^{\text{tr}} + \lambda_{\text{rot}} L_{\text{denoise}}^{\text{rot}} + \lambda_{\text{grip}} L_{\text{denoise}}^{\text{grip}}. \quad (9)$$

We use equal coefficient-denoising weights for all SMILE variants, setting $\lambda_{\text{tr}} = \lambda_{\text{rot}} = \lambda_{\text{grip}} = 1$.

D. Coefficient-target construction

We construct coefficient targets from the same expert action trajectories used by each baseline. For every eligible start step s , we collect the next F native actions, convert translation and rotation into cumulative command-space trajectories, and retain the gripper commands in their absolute form. For each modality $m \in \{\text{tr}, \text{rot}, \text{grip}\}$, the target coefficients are obtained by least-squares projection onto the fixed B-spline basis:

$$\Theta_s^{m,*} = \arg \min_{\Theta} \left\| \Phi_F^{(P,K)} \Theta - Y_s^m \right\|_F^2, \quad (10)$$

where Y_s^m denotes the corresponding cumulative or absolute target trajectory. We compute dataset-level coefficient statistics for normalization and apply validity masks near episode boundaries.

In summary, SMILE replaces raw per-step action denoising with coefficient-space denoising. Target coefficients are obtained by least-squares projection of transformed trajectories onto a fixed B-spline basis. At inference time, the denoised coefficients are decoded through the same basis and generate temporally coupled native controller actions.

V. EXPERIMENTS

We evaluate SMILE across four architecturally heterogeneous action experts on LIBERO, CALVIN, and real-world xArm environments, measuring task accuracy, execution horizon, amortized inference latency, and action smoothness.

A. LIBERO Benchmark

LIBERO contains four suites, Spatial, Object, Goal, and Long, comprising 40 language-conditioned manipulation tasks [27]. We evaluate SmolVLA and Evo1 over 2,000 episodes, using 50 episodes per task, and report suite-wise and overall success together with execution horizon, amortized inference latency, and model size. LIBERO-Long specifically evaluates temporally extended manipulation, where errors within an executed action chunk can compound across task stages. A representative SMILE-Evo1 success rollout is shown in Fig. 3.

Fig. 1b and Table I summarize the LIBERO accuracy–efficiency results. SMILE-Evo1 increases overall success from 92.7% to 98.0% while extending the execution horizon from 14 to 16 and reducing amortized latency from 8.5 to 8.0 ms per action. SMILE-Evo1 therefore establishes the strongest accuracy-efficiency operating point among the compared methods. SMILE-SmolVLA increases success from 74.0% to 82.6% while extending the horizon from 1 to 10 and reducing latency from 243.9 to 24.2 ms per action, corresponding to a $10.1\times$ speedup. Both integrations retain the reported model-size scales of their corresponding baselines.

The suite-level results show that SMILE-SmolVLA improves all four suites, with its largest gain on LIBERO-Long, from 42.8% to 63.6%. SMILE-Evo1 also improves Spatial, Object, and Long, reaching 98.2% on LIBERO-Long. These gains support the hypothesis that smoother action generation is particularly beneficial for temporally extended tasks requiring reliable execution over longer fixed horizons.

TABLE I: SMILE variants improve accuracy with longer fixed execution horizons at comparable model scale on LIBERO. Inf./action denotes amortized inference latency per executed action.

Model	Exe. Horizon $\uparrow$	Spatial $\uparrow$	Object $\uparrow$	Goal $\uparrow$	Long $\uparrow$	All $\uparrow$	Inf./action (ms) $\downarrow$	Model Size $\downarrow$
GR00T N1.7 [24]	8	97.2	96.8	97.8	93.4	96.3	14.7	3B
π_0 [16]	5	96.8	98.8	95.8	85.2	94.2	69.4	3.3B
OpenVLA [22]	1	84.7	88.4	79.2	53.7	76.5	288.2	7.5B
OpenVLA-OFT [23]	8	97.7	98.0	96.1	96.1	96.8	22.3	7.7B
SmolVLA [3]	1	78.8	91.4	82.8	42.8	74.0	243.9	0.4B
SMILE-SmolVLA (Ours)	10	84.8	95.0	86.8	63.6	82.6	24.2	0.4B
Evo1 [5]	14	88.4	94.2	98.2	89.8	92.7	8.5	0.8B
SMILE-Evo1 (Ours)	16	97.2	99.6	96.8	98.2	98.0	8.0	0.8B

Unlike the raw-action trade-off in Section III, these paired comparisons improve accuracy and inference efficiency simultaneously, without increasing model scale. They therefore isolate the effect of replacing raw per-step action generation with compact B-spline coefficient prediction.

B. CALVIN Benchmark

CALVIN evaluates long-horizon language-conditioned behavior through sequences of five consecutive subtasks [28]. We follow the standard ABC $\rightarrow$ D protocol used in prior evaluations [4], [6]. We evaluate SMILE with VPP and DAWN to test its applicability to predictive-visual and explicit pixel-motion-conditioned denoising action experts.

Table II extends the evaluation to predictive-visual and explicit pixel-motion-conditioned action experts. Both SMILE variants increase the execution horizon from 10 to 15 while improving average completed sequence length. SMILE-DAWN reduces amortized latency from 32.0 to 22.7 ms per action and increases average sequence length from 4.10 to 4.18, while SMILE-VPP reduces latency from 19.1 to 13.0 ms and increases average sequence length from 4.33 to 4.42. Most gains occur in later subtasks, where errors are more likely to accumulate across the sequence. Because the corresponding model-size scales remain unchanged, these results demonstrate that SMILE generalizes beyond VLM-centered action experts.

SMILE-VPP reaches an average sequence length of 4.42, only 0.02 below DreamVLA’s 4.44, while reducing amortized inference latency from 193.2 to 13.0 ms per action relative to DreamVLA, a 14.9 $\times$ speedup. This comparison places SMILE-VPP at a strong accuracy–efficiency operating point on CALVIN.

C. Real-world Experiment

a) Implementation: We use a single-arm setup with a 7-DoF xArm7 and two RGB cameras: a fixed third-person camera and a gripper-mounted camera. We compare Evo1 and VPP with their corresponding SMILE variants. Each matched pair is initialized from the same best-performing official pretrained checkpoint and fine-tuned on our collected dataset for the same number of optimization steps, providing a matched fine-tuning budget. The dataset contains 1,000 episodes across 12 tasks.

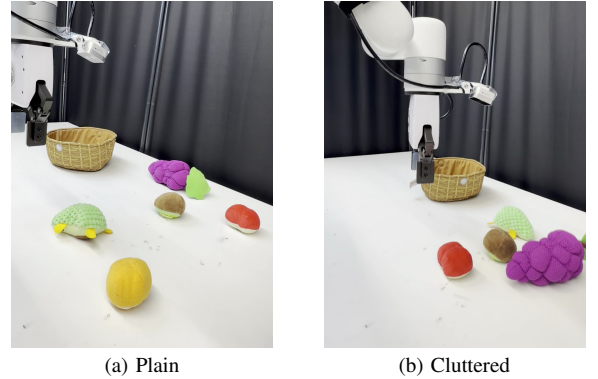


Fig. 4: Representative xArm kiwi-task layouts. The plain setting uses four separated distractors, whereas the cluttered setting places three distractors tightly around the kiwi.

b) Evaluation: The robot receives a language instruction to lift a specified object and place it into a basket. We evaluate six object categories under two arrangements, yielding 12 object-setting tasks. Fig. 4 shows representative configurations for the prompt “lift the kiwi and place it in the basket.” In the *plain* setting, four distractors surround the target with sufficient free space. In the *cluttered* setting, three nearby distractors tightly surround the target, thereby testing accurate approach, grasping, and transport without contacting neighboring objects.

Each task is evaluated over 20 episodes with randomized initial object configurations. We use $H = 6$ for Evo1 in the real-world setup rather than the LIBERO setting of $H = 14$, because $H = 6$ was the longest tested horizon that produced sufficiently reliable xArm rollouts.

c) Results: SMILE improves success for every object category in both settings. In the plain setting, average success increases from 26.7% to 78.3% for Evo1 and from 82.5% to 98.3% for VPP. In clutter, it increases from 8.3% to 56.7% and from 70.8% to 88.3%, respectively. Fig. 5 shows a representative SMILE-VPP cluttered-kiwi success, progressing from approach to pickup and basket placement. The VPP variants achieve higher absolute success than their Evo1 counterparts, plausibly reflecting VPP’s larger model scale and broader real-world pretraining. We therefore emphasize each matched baseline-to-SMILE comparison.

TABLE II: CALVIN ABC→D long-horizon evaluation. Columns 1–5 report success rates on consecutive subtasks, and Avg. Len. denotes the mean number completed. SMILE variants use longer execution horizons while improving average sequence length. Inf./action denotes amortized inference latency per executed action.

Model	Exe. Horizon ↑	1 ↑	2 ↑	3 ↑	4 ↑	5 ↑	Avg. Len. ↑	Inf./action (ms) ↓	Model Size ↓
VidMan [26]	1	0.915	0.764	0.682	0.592	0.467	3.42	295.0	0.7B
Seer [25]	1	0.944	0.872	0.799	0.722	0.643	3.98	88.5	0.3B
Seer-Large [25]	1	0.963	0.916	0.861	0.803	0.740	4.28	150.6	0.6B
DreamVLA [18]	1	0.982	0.946	0.895	0.834	0.781	4.44	193.2	0.8B
DAWN [6]	10	0.978	0.916	0.813	0.752	0.641	4.10	32.0	1.2B
SMILE-DAWN (Ours)	15	0.972	0.904	0.827	0.769	0.711	4.18	22.7	1.2B
VPP [4]	10	0.965	0.909	0.866	0.820	0.769	4.33	19.1	1.8B
SMILE-VPP (Ours)	15	0.962	0.925	0.885	0.846	0.798	4.42	13.0	1.8B

TABLE III: Real-world xArm lift-and-place results. Succ. and W.Obj. report lift→placement counts for the correct and wrong objects, respectively. Hit denotes distractor-contact episodes, and F. denotes failures to pick up the target within three attempts, workspace exits, or freezes. Aggregate columns report execution horizon (E.H.), amortized inference latency per action (I/a), success rate (SR), wrong-object pickup rate (WPR), drop rate after any pickup (DR), and hit rate (HR) over all six objects.

Setting	Model	All objects						Apple				Avocado				
		E.H.↑	I/a↓	SR↑	WPR↓	DR↓	HR↓	Succ.↑	W.Obj.↓	Hit↓	F.↓	Succ.↑	W.Obj.↓	Hit↓	F.↓	
Plain	Evo1	6	21.3	26.7%	30.0%	10.0%	27.5%	6→5	7→5	5	2	9→8	4→4	4	2	
	SMILE-Evo1	16	8.0	78.3%	16.7%	0.0%	4.2%	16→16	4→4	0	0	19→19	1→1	0	0	
	VPP	10	19.1	82.5%	0.8%	5.8%	6.7%	18→16	0	0	2	18→18	1→1	1	0	
	SMILE-VPP	15	13.0	98.3%	0.0%	0.0%	0.0%	20→20	0	0	0	20→20	0	0	0	
Cluttered	Evo1	6	21.3	8.3%	44.2%	8.3%	45.0%	2→2	11→9	7	0	2→2	10→8	6	2	
	SMILE-Evo1	16	8.0	56.7%	20.8%	0.0%	22.5%	8→8	10→10	2	0	18→18	0	2	0	
	VPP	10	19.1	70.8%	0.8%	3.3%	25.0%	12→11	0	8	0	17→17	0	3	0	
	SMILE-VPP	15	13.0	88.3%	0.0%	0.0%	11.7%	15→15	0	5	0	19→19	0	1	0	
Model	Banana				Grape				Kiwi				Orange			
	Succ.↑	W.Obj.↓	Hit↓	F.↓	Succ.↑	W.Obj.↓	Hit↓	F.↓	Succ.↑	W.Obj.↓	Hit↓	F.↓	Succ.↑	W.Obj.↓	Hit↓	F.↓
Evo1	8→6	4→3	6	2	7→6	4→4	8	1	4→4	8→6	6	2	4→3	9→8	4	3
SMILE-Evo1	18→18	1→1	0	1	16→16	1→1	3	0	13→13	5→5	2	0	12→12	8→8	0	0
VPP	14→12	0	6	0	18→17	0	1	0	18→18	0	0	2	20→18	0	0	0
SMILE-VPP	20→20	0	0	0	19→19	0	0	1	20→20	0	0	0	19→19	0	0	1
Evo1	3→2	5→4	12	0	3→3	3→2	14	0	1→1	11→10	8	0	0	13→11	7	0
SMILE-Evo1	12→12	2→2	6	0	13→13	0	7	0	10→10	4→4	6	0	7→7	9→9	4	0
VPP	13→12	0	7	0	18→17	0	2	0	16→16	1→1	3	0	13→12	0	7	0
SMILE-VPP	18→18	0	2	0	19→19	0	1	0	18→18	0	2	0	17→17	0	3	0

In clutter, SMILE-Evo1 and SMILE-VPP reduce hit rate by 22.5 and 13.3 percentage points (pp), respectively, and reduce wrong-object pickup rate by 23.4 and 0.8 pp. These higher baseline error rates can both reflect and compound distribution shift: jittery actions can move the robot and scene away from states represented in the demonstrations, leading to contact with distractors or wrong-object grasps. These failures

can then further alter the scene and degrade subsequent action predictions.

Qualitative rollouts also show more stable gripper behavior. The baseline policies sometimes oscillate between opening and closing commands and release an object during transport; Fig. 6 shows a VPP example in which the orange drops before reaching the basket center. Both SMILE variants eliminate

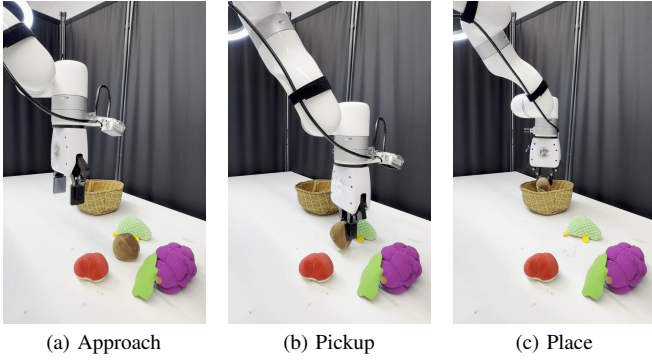


Fig. 5: SMILE-VPP cluttered-kiwi success: approach, pickup, and placement.

the aggregate drops observed for the baselines, reducing drop rate in the plain setting from 10.0% and 5.8% to 0.0% for Evo1 and VPP, respectively.

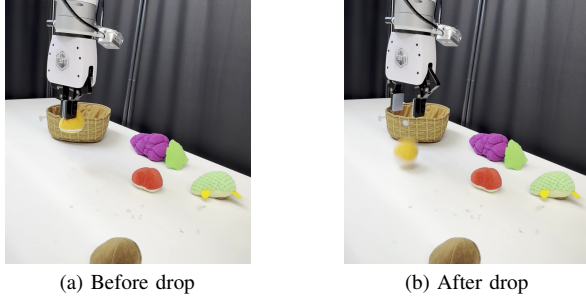
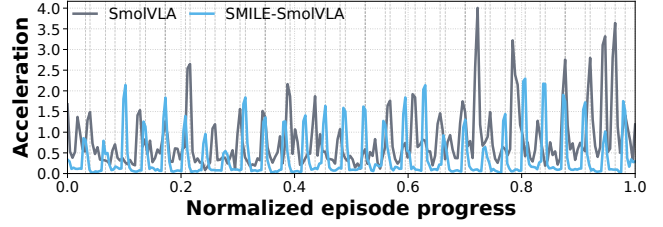


Fig. 6: VPP orange-drop failure: the orange is held near the basket in (a) but released before reaching the basket center in (b).

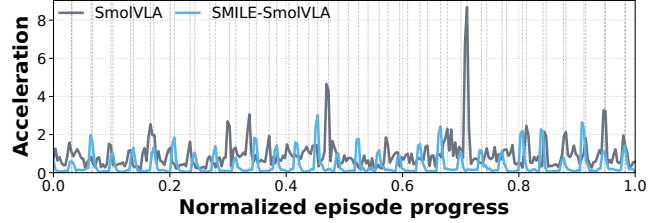
SMILE increases the executed horizon from 6 to 16 for Evo1 and from 10 to 15 for VPP, yielding $2.7\times$ ($21.3\rightarrow 8.0$ ms) and $1.5\times$ ($19.1\rightarrow 13.0$ ms) speedups, respectively. These results verify that smoother actions improve both task accuracy and inference efficiency in simulation and real-world environments.

D. Action Smoothness Analysis

To examine the mechanism behind SMILE’s accuracy–efficiency gains, we compare SmoIVLA and SMILE-SmoIVLA on LIBERO at the same execution horizon, $H = 10$, thereby matching the model-refresh frequency and isolating the effect of the action representation. All metrics are computed from normalized translation and rotation actions. For an executed sequence $a_{1:T}$, we define finite-difference acceleration as $\Delta^2 a_t = a_{t+1} - 2a_t + a_{t-1}$. We average the per-episode 95th percentile of $\|\Delta^2 a_t\|$ across episodes, reporting global acceleration, non-boundary acceleration within chunks, and boundary acceleration near model-refresh transitions. We additionally report the velocity sign-change rate (Vel. ZCR), defined as the fraction of adjacent velocity steps that reverse sign, as a measure of oscillatory motion.



(a) Task 3: put the black bowl in the bottom drawer of the cabinet and close it.



(b) Task 7: put both the alphabet soup and the cream cheese box in the basket.

Fig. 8: Acceleration traces for two LIBERO-10 tasks at $H = 10$; dashed lines mark model-call boundaries.

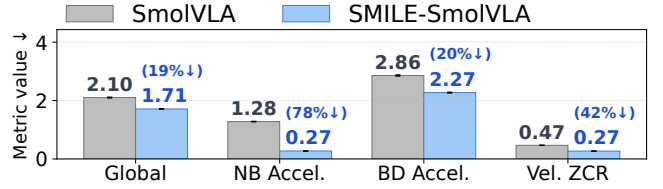


Fig. 7: Aggregate LIBERO smoothness metrics. Lower values indicate smoother execution.

As shown in Fig. 7, SMILE-SmoIVLA reduces non-boundary acceleration by 78.6% and Vel. ZCR by 42.3%, while reducing global and boundary acceleration by 18.6% and 20.5%, respectively. The much larger reduction within chunks indicates that coefficient-space generation primarily suppresses intra-chunk jitter, rather than only smoothing transitions between model calls.

The traces in Fig. 8 show frequent high-amplitude acceleration spikes for SmoIVLA, both within chunks and near model-call boundaries. SMILE-SmoIVLA maintains a lower acceleration baseline and suppresses high-frequency fluctuations across both task types. This qualitative evidence agrees with the aggregate metrics and supports smoother chunk execution as the mechanism behind improved long-horizon reliability.

E. Ablation Study

This section evaluates SMILE’s robustness across multiple execution horizons and identifies the optimal B-spline hyperparameters.

Fig. 9 shows that SMILE-SmoIVLA and SMILE-Evo1 consistently outperform their corresponding raw-action baselines across multiple long execution horizons. The benefit of coefficient-space action generation is therefore robust to the

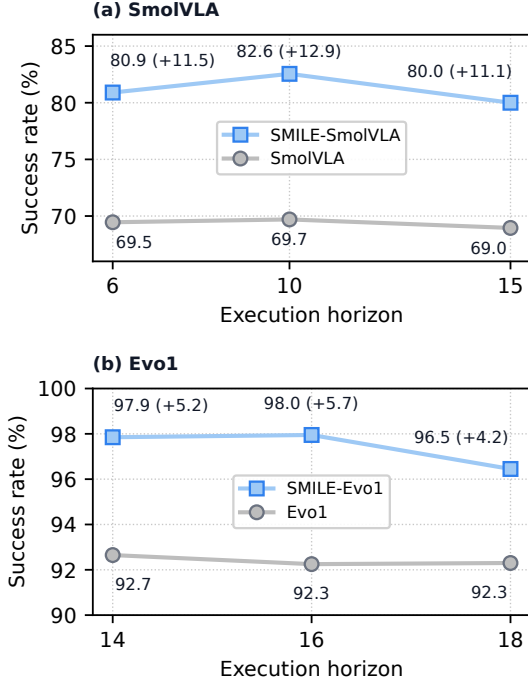


Fig. 9: LIBERO success versus execution horizon. SMILE outperforms raw-action baselines across tested horizons.

TABLE IV: SMILE-SmolVLA spline ablations on LIBERO. Highlighted cells mark selected hyperparameters.

(a) Control points ($P = 3$).

K	6	8	10
Succ.(%)	80.05	82.55	80.75

(b) Spline degree ($K = 8$).

P	3	5	7
Succ.(%)	82.55	81.90	81.75

horizon choice rather than arising from a single favorable operating point.

Table IVa shows that, with cubic splines, $K = 8$ control points performs best. Table IVb shows that, with $K = 8$ fixed, the cubic degree $P = 3$ outperforms the higher-degree alternatives. A modest low-order basis is therefore sufficient; increasing spline complexity does not improve success and may weaken the useful smoothness bias.

VI. CONCLUSION

We introduced *SMILE*, an architecture-preserving B-spline action interface that replaces raw per-step action generation with compact coefficient-space prediction. By improving the temporal consistency of predicted action chunks, *SMILE* enables longer fixed-horizon execution with higher task accuracy and lower amortized inference latency per action. Its consistent improvements across heterogeneous VLA action experts, simulation benchmarks, and real-world manipulation demonstrate that smooth coefficient-action generation is a general mechanism for improving the accuracy–efficiency trade-off of compact VLAs, without replacing their original visual-language conditioning or denoising backbones.

REFERENCES

- [1] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Proc. Robot.: Sci. Syst. (RSS)*, Daegu, Republic of Korea, July 2023.
- [2] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. C. M. Burchfiel, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Proc. Robot.: Sci. Syst. (RSS)*, Daegu, Republic of Korea, July 2023.
- [3] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zoutine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene, “SmolVLA: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025.
- [4] Y. Hu, Y. Guo, P. Wang, X. Chen, Y.-J. Wang, J. Zhang, K. Sreenath, C. Lu, and J. Chen, “Video prediction policy: A generalist robot policy with predictive visual representations,” in *Proc. 42nd Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 267, July 2025, pp. 24 328–24 346.
- [5] T. Lin, Y. Zhong, Y. Du, J. Zhang, J. Liu, Y. Chen, E. Gu, Z. Liu, H. Cai, Y. Zou, L. Zou, Z. Zhou, G. Li, and B. Zhao, “Evo-1: Lightweight vision-language-action model with preserved semantic alignment,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2026.
- [6] E.-R. Nguyen, Y. Zhang, K. Ranasinghe, X. Li, and M. S. Ryoo, “Pixel motion diffusion is what we need for robot control,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2026.
- [7] Y. Liu, J. I. Hamid, A. Xie, Y. Lee, M. Du, and C. Finn, “Bidirectional decoding: Improving action chunking via guided test-time sampling,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.
- [8] Y. Liang, X. Wang, K. Wang, S. Wang, X. Peng, H. Chen, D. K. H. Chua, and P. Vadakkepatt, “Adaptive action chunking at inference-time for vision-language-action models,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2026.
- [9] H. Wang, G. Zhang, Y. Yan, R. R. Kompella, and G. Liu, “VLA knows its limits: Adaptive execution horizons for robot policies,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2026.
- [10] K. Black, M. Y. Galliker, and S. Levine, “Real-time execution of action chunking flow policies,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2025.
- [11] F. Yang, P. Jing, K. Qu, N. Zhao, and Y. Su, “ABPolicy: Asynchronous B-spline flow policy for real-time and smooth robotic manipulation,” *arXiv preprint arXiv:2602.23901*, 2026.
- [12] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine, “FAST: Efficient action tokenization for vision-language-action models,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2025.
- [13] H. Zhou, W. Liao, X. Huang, Y. Tang, F. Otto, X. Jia, X. Jiang, S. Hilber, G. Li, Q. Wang, Ö. E. Yağmurlu, N. Blank, M. Reuss, and R. Lioutikov, “BEAST: Efficient tokenization of B-splines encoded action sequences for imitation learning,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2025.
- [14] X. Han, H. Xiong, H. Chen, C. Liu, A. Torralba, Y. Zhu, and Y. Du, “B-spline policy: Accelerating manipulation policies via b-spline action representations,” *arXiv preprint arXiv:2607.09648*, 2026.
- [15] M. Tian, Y. Li, S. Liu, A. Ijspeert, and S. Calinon, “Spline policy: A structured representation for robot policies,” *arXiv preprint arXiv:2606.07386*, 2026.
- [16] K. Black, N. Brown, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, L. Smith, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky, “ π_0 : A vision-language-action flow model for general robot control,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2025.
- [17] J. Wen, Y. Zhu, J. Li, Z. Tang, C. Shen, and F. Feng, “DexVLA: Vision-language model with plug-in diffusion expert for general robot control,” in *Proc. Conf. Robot Learn. (CoRL)*, 2025.
- [18] W. Zhang, H. Liu, Z. Qi, Y. Wang, X. Yu, J. Zhang, R. Dong, J. He, H. Wang, Z. Zhang, L. Yi, W. Zeng, and X. Jin, “DreamVLA: A vision-language-action model dreamed with comprehensive world knowledge,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2025.
- [19] C. Zhu, R. Yu, S. Feng, B. Burchfiel, P. Shah, and A. Gupta, “Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2025.
- [20] Y. Shen, F. Wei, Z. Du, Y. Liang, Y. Lu, J. Yang, N. Zheng, and B. Guo, “VideoVLA: Video generators can be generalizable robot manipulators,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2025.

- [21] J. Chen, W. Song, P. Ding, Z. Zhou, H. Zhao, F. Tang, D. Wang, and H. Li, “Unified diffusion VLA: Vision-language-action model via joint discrete denoising diffusion process,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.
- [22] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “OpenVLA: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [23] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2025.
- [24] NVIDIA, “NVIDIA Isaac GR00T N1.7: Open reasoning VLA model for humanoid robots,” NVIDIA Isaac GR00T model release, 2026.
- [25] Y. Tian, S. Yang, J. Zeng, P. Wang, D. Lin, H. Dong, and J. Pang, “Predictive inverse dynamics models are scalable learners for robotic manipulation,” *arXiv preprint arXiv:2412.15109*, 2024.
- [26] Y. Wen, J. Lin, Y. Zhu, J. Han, H. Xu, S. Zhao, and X. Liang, “VidMan: Exploiting implicit dynamics from video diffusion model for effective robot manipulation,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024.
- [27] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” *arXiv preprint arXiv:2306.03310*, 2023.
- [28] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard, “CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks,” *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 7327–7334, July 2022.